PRÁCTICA 1:
DOCKERIZACIÓN DE APLICACIONES MODERNAS

Resumen

En esta práctica se van a afianzar los conceptos aprendidos en la primera parte de esta. Así, en primer lugar, se creará una imagen de un programa sencillo que imprima por pantalla "Hola Mundo" y se ejecutará en un contenedor. Después, se hará lo mismo con una aplicación distribuida, consistente en un frontal web y una Application Programming Interface (API) proporcionada por los profesores. Finalmente, se comprobará la conexión distribuida entre ambos componentes.

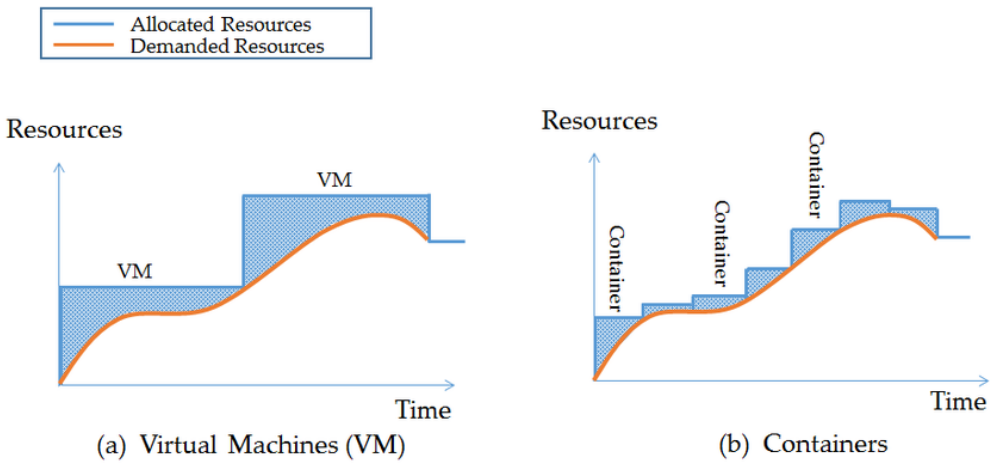
1. Objetivos

- Comprender los fundamentos de la "contenerización" de aplicaciones con Docker.
- Implementar una arquitectura distribuida básica.
- Gestionar comunicación entre servicios distribuidos basados en recursos.

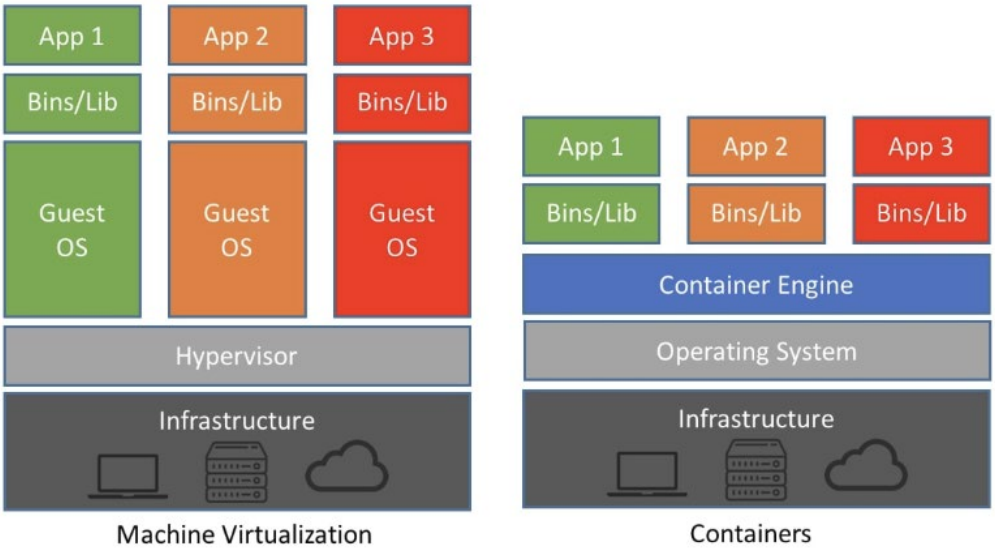
2. Fundamentos teóricos

Contenedores

La *contenerización* es una técnica de desarrollo en la que una aplicación, junto con sus dependencias y configuración, se empaqueta en una *imagen de contenedor*. Esto permite que la aplicación pueda ser probada y desplegada fácilmente en diferentes entornos, funcionando siempre de la misma manera. Los contenedores sirven como una unidad estándar de empaquetado y despliegue de software, lo que facilita que los desarrolladores desplieguen aplicaciones en distintos sistemas sin apenas modificaciones. Además, los contenedores aíslan las aplicaciones entre sí dentro de un mismo sistema operativo, y su consumo de recursos es mucho menor que el de las máquinas virtuales (Figura 1). Cada contenedor puede ejecutar una aplicación web completa o un servicio específico.



(a) Virtual Machines (VM) (b) Containers



3. Prerrequisitos

Es necesario asegurarse de que *Docker Desktop* está instalado en la máquina. Si no es así, se pueden seguir los pasos de la primera parte de la Práctica 1 para instalarlo y conocer su uso básico.

Entorno de Desarrollo

Para el desarrollo de las prácticas de la asignatura se empleará Microsoft Visual Studio Code como entorno de desarrollo integrado (IDE). VSCode es ligero y gratuito y funciona en Windows, MacOS y Linux, ofreciendo soporte para prácticamente cualquier lenguaje de programación. En el caso concreto de la asignatura, se desarrollará en lenguaje C# sobre la plataforma .Net 10.

4. Desarrollo

En esta práctica se han planteado 3 ejercicios.

Ejercicio 1. Hola Mundo con contenedores

En este ejercicio se pide crear una aplicación de consola en .Net que imprima por pantalla el texto "Hola Mundo". Además, se requiere crear la imagen de dicha aplicación y desplegar dicha imagen en un contenedor. Los pasos a seguir son los siguientes:

Proyecto Hola Mundo

1.- En la terminal, crea un proyecto de consola con el comando:

```
dotnet new console Hello-World
```

Y añade el código necesario.

Dockerización de la aplicación

1.- En la misma carpeta donde creaste la aplicación de consola crea un fichero Dockerfile (sin extensión) con el siguiente contenido:

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /HelloWorld

# Copy everything
COPY . ./

# Restore as distinct layers
RUN dotnet restore

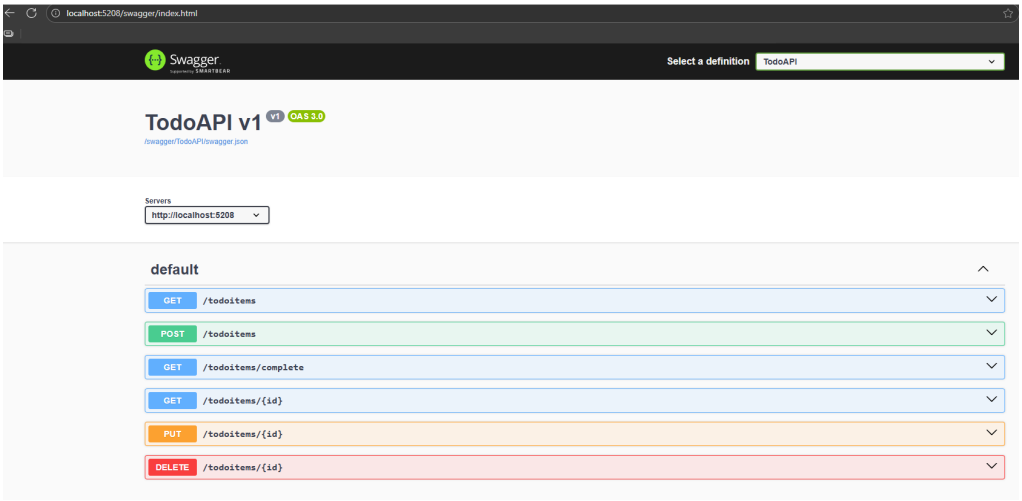
# Build and publish a release
RUN dotnet publish -o out

# Build runtime image
FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /HelloWorld
COPY --from=build /HelloWorld/out .
ENTRYPOINT ["dotnet", "HelloWorld.dll"]
```

2.- Crea una imagen de Docker para esta aplicación basándote en los conocimientos adquiridos en la parte 1 de esta práctica.
3.- Una vez creada la imagen, ejecuta dicha imagen en un contenedor y comprueba que la salida por pantalla es "Hola Mundo".

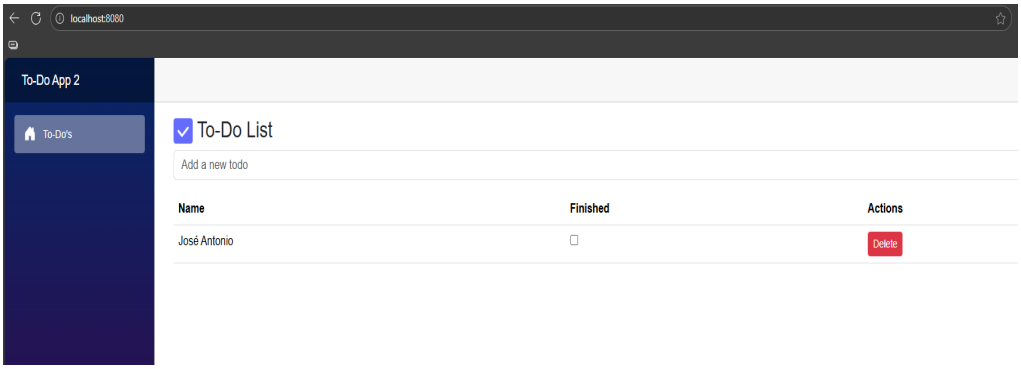
Ejercicio 2. Application Programming Interface (API)con contenedores

1.- Descarga del campus virtual de la asignatura el fichero "TodoApi.zip" y descomprímelo.
2.- Crea un fichero Dockerfile para este proyecto que permita crear una imagen de esta aplicación. Debes basarte en el anterior y recuerda añadir una línea que exponga (EXPOSE) el puerto 8080 en el contenedor.
3.- Crea una imagen de dicho proyecto y despliega un contenedor con dicha imagen. Cuando despliegues el contenedor debes hacer un mapeo de puertos entre el puerto 8080 del contenedor y el 5208 del host. Este puerto se puede modificar en función de vuestra máquina, pero recordad cambiarlo en el ejercicio 4 para que el frontal web apunte a la API correctamente. Para comprobar que el funcionamiento es correcto, debes entrar en la URL de la API en tu host (TodoAPI):



Ejercicio 3. Frontal web con contenedores

1.- Descarga del campus virtual de la asignatura el fichero "TodoApp.zip" y descomprímelo.
2.- Modifica el fichero Dockerfile para este proyecto que permita crear una imagen de esta aplicación. Debes basarte en el anterior (sigue los comentarios que hay en el fichero) y no olvides añadir una línea que exponga (EXPOSE) el puerto 80 (HTTP) en el contenedor.
3.- Crea una imagen de dicho proyecto y despliega un contenedor con dicha imagen. Cuando despliegues el contenedor debes hacer un mapeo de puertos entre el puerto 80 del contenedor y el 8080 del host (puede ser que este puerto esté en uso en tu PC, por lo que puedes usar otro como, por ejemplo, el 8081). Este puerto se puede modificar en función de vuestra máquina, pero recordad cambiarlo en el ejercicio 4 para que el frontal web apunte a la API correctamente. Para comprobar que el funcionamiento es correcto, debes entrar en la URL del frontal en tu host (TodoApp.Client):



Ejercicio 4. Aplicación web distribuida [Opcional]

En esta parte de la práctica se va a crear una aplicación distribuida basándonos en el frontal y la API de los apartados anteriores. En este caso, uno de los alumnos va a desplegar la API y los otros el frontal. Los pasos a seguir son los siguientes:

1.- Averigua la IP de tu máquina. En Windows, el comando *ipconfig* te será de utilidad mientras que en Linux puedes usar *ip addr*.
2.- Los alumnos/as que vayan a desplegar el frontal deben apuntar dicha aplicación a la API. Para ello, deben cambiar la dirección base en el cliente *http* del fichero

Program.cs del proyecto de la aplicación TodoApp para que apunte a la IP que habéis obtenido en el punto anterior.

3.- Una vez hecho esto, redesplegáis el frontal y comprobáis si la aplicación funciona.

4.- Lo normal es que os encontréis con un problema de **CORS** (***Cross-Origin Resource Sharing*** o en español Intercambio de recursos de origen cruzado) (¿Qué es CORS? - Explicación del uso compartido de recursos entre orígenes - AWS). Por tanto, debéis configurar la API para que acepte las peticiones del frontal. Para ello, quien vaya a desplegar la API debe hacer ciertas modificaciones en el fichero Program.cs del proyecto TodoApi. Una vez, hechas las modificaciones debéis regenerar la imagen y redesplegar el contenedor. Finalmente, si todo funciona, en el frontal podréis añadir tareas a la lista de *To-do*.